화면 및 컴포넌트 설계 (React 기반)

```
.
├── App.tsx
├── features : 도메인 기능 폴더
│   ├── auth
│   │   ├── _apis : api 호출 함수
│   │   ├── _components : 컴포넌트
│   │   ├── _constants : 상수
│   │   ├── _hooks : react-query hook, customo 훅
│   │   ├── _mocks : mock api 함수
│   │   ├── _schemas : valid 검증 스키마
│   │   ├── _stores : 클라이언트 전역 state 저장소
│   │   ├── _types : 타입
│   │   └── index.ts
│   ├── category
│   ├── comment
│   ├── event
│   ├── group
│   ├── image
│   ├── notice
│   ├── post
│   └── users
├── libs : 유틸 함수
├── main.tsx : 진입점
├── mocks : mock api 설정
├── pages : 페이지 폴더
├── router : 라우팅
├── shared
│   ├── components : 공통 컴포넌트
│   ├── constants : 공통 상수, queryKey
```

```
│   ├── hooks : 공통 훅
│   ├── setup : 공통 setup
│   ├── stores : 클라이언트 zustand 저장소
│   ├── styles : 공통 스타일 코드
│   ├── types : 공통 타입
└── vite-env.d.ts
```

📘 React 프로젝트 및 컴포넌트 설계서

1. 프로젝트 구조 개요

- **기반 스택**: React + TypeScript + Vite
- **상태 관리**: Zustand(클라이언트 상태) + React Query(서버 상태)
- **데이터 통신**: React Query + Axios
- **스타일링**: SCSS Modules + Global Reset + Theme Color/Mixin
- **Mocking**: MSW (Mock Service Worker)
- **라우팅**: React Router v6
- 스타일 코드 규칙 : lint, prettier
- 기타 사용 라이브러리: zod, dayjs, lodash, clsx, msw

---

2. 강점

- **React Query + Axios** : 인터셉터(response, request 부분) 네트워크/토큰 관리 일원화, 요청에 따른 에러처리, 조회 부분 캐싱 전략
- **Zustand** : 전역 UI 상태 심플 관리 (Toast/Loading)
- **SCSS + Mixins** : 확장성 있는 스타일 시스템
- **MSW** : 서버 없는 환경에서도 독립 개발 가능
- **Zod Schema** : 클라이언트 레벨 폼 검증 통일
- 무한스크롤링, 페이지네이션 기능 제공

- typescript로 타입 확장, 재사용성, 런타임전 오류 감지
- 반응형 디자인 (모바일, 테블릿, 데스크탑 화면 지원)

---

3. 주요 폴더

- **features/**: 도메인 단위 기능 (auth, category, group, image, notice, comment, event, users, post 등)
  - _apis : api 요청 함수 모음
  - _components : 도메인 ui 컴포넌트
  - _hooks: react-query 요청 훅, 도메인 훅
  - _mocks : 가짜 api 함수
  - _schemas: form 검증(valid)
  - _types: response, query, body, 컴포넌트 type 정의
- **pages/**: 라우트 페이지 (main, login, signup, group, event, mypage, search, notfound, 등록/수정페이지 등)
- **shared/**: 공통 ui 컴포넌트, constant, hooks, store, styles, setup, types
- **libs/**: axios wrapper(apiCall), cookie, image, string, time 유틸
- **mocks/**: MSW 기반 mock 서버 설정

---

4. 전역 설정

- **App.tsx**
  - React Query Provider + Devtools 적용
  - 전역 Toast + Loading 연결 (`ui.store.ts`)
- **router.tsx**
  - 라우팅 전담
  - 인증 후 접근할 페이지 (`/main`, `/mypage`, `/group/...`) 구성
- **reactQuery.ts**

    - 전역 QueryClient 세팅
    - 인증 오류 시 자동 토큰 리셋 및 `/`(로그인) 리다이렉트
- **apiCall.ts**
    - Axios 인스턴스
    - AccessToken 자동 삽입, RefreshToken 기반 재발급 처리
    - Refresh api 요청 중엔 요청 큐(pendingRequests)에 저장 후 순차 처리

---

5. Shared

- **components/**: Button, InputField, Modal, Toast, Pagination, Layout 등 공통 UI
- **stores/ui.store.ts**:
    - `toast`, `loading` 상태 관리 (Zustand)
    - `showToast`, `showLoading`/`hideLoading` API 제공
- **hooks/**: 공통 훅 정의
    - `useIsMobile` : 모바일 화면인지 js에서 체크
    - `useLoading` : props로 loading 여부를 받고 loading ui 처리
    - `useQueryParams` : query parameter 함수 제공 훅
- **utils/**:
    - `time.ts`: 상대 시간 (`n분 전`)
    - `str.ts`: 숫자 추출, 포맷팅 유틸
- constants/:
    - queryKey: server 전역 state 캐싱에 필요한 키값 저장
- **styles/**:
    - `_color.scss`, `_mixin.scss`, `global.scss` → 테마 + Reset + Layout mixin

---

6. 페이지 설계

1) LoginPage

- 이메일 / 비밀번호 입력 + 검증
- `postLoginApi` → 성공 시 `setToken` 후 `/main` 이동
- 실패 시 에러 메시지 표시
- 하단에서 `/signup` 이동 버튼 제공

---

2) SignupPage

- 이메일, 닉네임, 지역, 비밀번호 입력
- 비밀번호 확인 + 개인정보처리방침 동의 체크
- `postSignupApi` → 성공 시 자동 로그인 후 `/main` 이동
- 실패 시 에러/Toast 처리

---

3) MainPage

- 검색창 (`SearchInput + Button`)
- `FilterList` 표시
- 내가 찜한 모임 리스트 (`useGetMyLikedGroups`)
- 추천 모임 리스트 (`useGetGroupsListApi`)
- `GroupSearchItem` 컴포넌트로 그룹 목록 표시
- `EmptyView`로 비어있는 상태 표시

---

4) MainSearch

- 쿼리스트링(`word`, `region`, `cate`, `page`) 기반 검색
- 그룹 리스트 + 페이지네이션
- `SectionTitle("'검색어'으로 검색한 내용")`
- `GroupSearchItem` 리스트 출력

---

5) MyGroupTabPage

- 다가오는 정모 일정 (`useGetMyUpcomingEvents`)
- 내가 참여한 모임 (`useGetMyJoinedGroupsApi`)

- `EventItem` + `GroupSearchItem` 렌더링
- 비어있을 경우 `EmptyView` 표시

---

6) GroupInfoPage

- 그룹 상세 정보 (`useGetGroupsOneApi`)
- `GroupBanner` + `GroupInfoContent`
- 일정 리스트 (`useGetEventsListApi`)
- 멤버 리스트 (`useGetGroupMemberApi`)
- 일정 참여/취소(`usePostEventsAttendeeApi`, `useDeleteCancelEventAttendeeApi`)
- ActionSheet로 참석/취소/수정 처리
- 모임장은 "+ 일정 생성" 버튼 가능

---

7) GroupSettingPage

- 그룹장 전용 설정 페이지
- 멤버 강퇴 / 위임 (`useDeleteGroupsMemberApi`, `usePatchDelegateOwner`)
- 그룹 삭제 (`useDeleteGroupsApi`)
- 일정 삭제 (`useDeleteEventEventsApi`)
- 이벤트 참석자 관리 모달 (`EventAttendeesModal`)

---

8) GroupRegisterPage

- 모임 신규 등록 / 수정 겸용
- `groupSchema` 검증 (Zod)
- 태그 추가/삭제 (최대 3개)
- 이미지 업로드 (`useUploadImage`)
- 성공 시 `/group` 이동

---

9) GroupPostPage

- 모임 게시판 목록 페이지
- 무한 스크롤링 커서 기반 조회 (`useGetPostsWithCursorApi`)

- 게시글 등록/수정/삭제 지원
- `PostContentModal` 로 상세 내용 보기
- ActionSheet → 수정/삭제 선택 가능
- 모임장은 "글 작성" 버튼 제공

---

10) **PostRegisterPage**

- 게시글 등록/수정 페이지
- `postSchema` 검증 (Zod)
- 이미지 업로드 지원
- 등록 성공 시 `/group/:groupId/post` 로 이동

---

11) **EventRegisterPage**

- 일정 등록 / 수정 겸용
- `eventSchema` 검증 (Zod)
- 이미지 업로드 (`useUploadImage`)
- 성공 시 Toast + 그룹 상세로 이동

---

12) **Mypage**

- 사용자 정보 + 프로필 (`MyProfileView`)
- 내부 탭: 찜한 모임 / 내 게시글 / 내 댓글
- 각 탭은 Pagination 지원
- 데이터: `useGetMyLikedGroups`, `useGetMyPosts`, `useGetMyComments`

---

13) **ProfileEditPage**

- 프로필 수정 (닉네임, 소개글, 지역, 프로필 이미지)
- `usePatchUser` API 연동
- 이미지 업로드 (`useUploadImage`)
- 성공 시 이전 화면으로 복귀 + Toast 알림

---

14) **SettingPage**

- 단순 설정 메뉴 제공
- 현재는 **공지사항** 이동 버튼만 존재

---

15) **NoticePage**

- 공지사항 리스트 (`useGetNotices`)
- 클릭 시 본문 토글 표시
- 날짜는 `dayjs` 포맷(`YYYY.MM.DD`)

---

16) **NotFoundPage**

- 404 전용 페이지
- 메시지 + 이스터에그(🥚) + "집으로 돌아가기" 버튼
- `/main`으로 이동 버튼 제공

---

7. 주요 데이터 흐름

1. **인증**
    - 로그인 성공 시 AccessToken 저장 (Zustand)
    - Axios 인터셉터로 자동 헤더 삽입
    - 만료 시 Refresh API 호출 → 실패 시 로그아웃 처리
2. **데이터 요청**
    - React Query 기반 API 요청
    - `useLoading` 훅으로 전역 로딩 관리
    - `useUiStore` 의 Toast로 에러 표시
3. **UI 상태**
    - Toast / Loading → 전역 관리
    - 각 페이지는 `CommonLayout` 으로 UI 일관성 유지